

EduCore AI

AI-Powered Education Management Platform
Implementation Plan — Hackathon Edition (2 Days)

■ Beginner
Friendly

■ 2-Day Build

■ AI + RAG
Chatbot

■ PDF Outputs

■ School	Al-Noor Academy
■ Build Time	2 Days (Hackathon)
■ Backend	FastAPI + SQLite + Python
■ Frontend	React + Vite + Tailwind CSS
■ AI Stack	scikit-learn + SHAP + FAISS + Groq
■ PDFs	Fee Challan + Report Card (ReportLab)
■ Auth	JWT (admin: admin / admin123)

Generated on September 12, 2026

1. Project Goal

Build a **demo-ready, AI-powered School Management System** in 48 hours that replaces manual Excel-based school administration with a centralized digital platform. Judges will see: real data flows, professional PDF outputs, machine-learning-based at-risk student predictions with explainability, and an AI-powered policy chatbot — all running live.

2. Project Scope

■ In Scope (Building)	■ Out of Scope (Removed)
Student registration & management	Attendance tracking
Fee management + PDF challan generation	WhatsApp / SMS alerts
Result entry + report card PDF	Parent portal
Admin dashboard with live charts	Timetable scheduling
At-risk AI prediction + SHAP explanations	Multi-tenancy (Phase 4)
RAG Policy Chatbot (Groq + FAISS)	
JWT secure admin login	

3. Tech Stack

Backend

Tool	Role	Beginner Note
Python + FastAPI	Server + API endpoints	Auto-generates /docs test page
SQLite	Database (single .db file)	Zero setup — no server needed
SQLModel	ORM — Python → DB tables	Write Python classes, get DB tables
ReportLab	Generate PDF files	Fee challans + report cards
scikit-learn	Machine learning	Trains at-risk prediction model
SHAP	Explain AI decisions	Plain English reasons per student
sentence-transformers	Text embeddings (free, local)	Converts policy text → numbers
FAISS	Vector similarity search	Finds relevant policy chunks
Groq API (free)	LLM — LLaMA 3	Generates chatbot answers

JWT	Secure auth tokens	Industry-standard login
-----	--------------------	-------------------------

Frontend

Tool	Role	Beginner Note
React + Vite	Web UI framework	Component-based, fastest dev setup
Tailwind CSS	Styling	Class names = styles, no CSS file
shadcn/ui	UI components	Ready-made buttons, tables, forms
React Router	Page navigation	Links between /dashboard, /students
Axios	HTTP requests	Talks to FastAPI backend
Recharts	Charts & graphs	Bar charts on dashboard

4. Project Folder Structure

```

G:\EduTech\
├──
├── ── backend\
│   ├── main.py           ← Start server here
│   ├── config.py         ← School name, API keys, settings
│   ├── database.py       ← SQLite connection
│   ├── seed_data.py      ← Load 50 demo students
│   ├── requirements.txt  ← Python packages list
│   ├──
│   ├── ── models\       ← Database table definitions
│   │   ├── user.py       ← Admin login accounts
│   │   ├── student.py    ← Student records
│   │   ├── fee.py        ← Invoices & payments
│   │   └── result.py     ← Exam marks
│   ├──
│   ├── ── routers\      ← API URL handlers
│   │   ├── auth.py       ← POST /auth/login
│   │   ├── students.py   ← CRUD /students
│   │   ├── fees.py       ← /fees – challan, pay, PDF
│   │   ├── results.py    ← /results – marks + report card
│   │   ├── dashboard.py  ← /dashboard/stats
│   │   ├── ai.py         ← /ai/at-risk predictions
│   │   └── chatbot.py    ← /chatbot/ask ← RAG
│   ├──
│   ├── ── services\     ← Business logic
│   │   ├── pdf_service.py ← Generate PDFs
│   │   ├── ai_service.py  ← At-risk ML model
│   │   └── rag_service.py ← RAG chatbot logic
│   ├──
│   ├── ── knowledge_base\ ← School policy text files
│   │   ├── fee_policy.txt
│   │   ├── exam_policy.txt
│   │   ├── admission_policy.txt
│   │   ├── conduct_rules.txt
│   │   └── academic_calendar.txt
│   ├──
├── ── frontend\
│   ├── ── src\
│   │   ├── pages\
│   │   │   ├── Login.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Students.jsx
│   │   │   ├── Fees.jsx
│   │   │   ├── Results.jsx
│   │   │   ├── AtRisk.jsx ← AI predictions
│   │   │   └── Chatbot.jsx ← RAG chatbot ← NEW
│   │   ├── components\
│   │   │   ├── Layout.jsx
│   │   │   ├── StatCard.jsx
│   │   │   └── DataTable.jsx

```

5. Database Schema

Table	Key Fields
users	id, username, password_hash, role
students	id, name, roll_number, grade, section, guardian_name, phone, dob, created_at
academic_years	id, label (e.g. '2025-2026'), is_current
fee_structures	id, grade, label, amount, academic_year_id
invoices	id, student_id, amount, due_date, challan_number, status (paid/pending)
payments	id, invoice_id, amount_paid, paid_at, payment_mode (cash/bank/online)
results	id, student_id, subject, marks_obtained, total_marks, exam_type, academic_year_id

■ The RAG chatbot uses flat .txt files + in-memory FAISS index — no extra DB table required.

6. API Endpoints

Auth

Method	Endpoint	Description
POST	/auth/login	Returns JWT token

Students

Method	Endpoint	Description
GET	/students	List all students (search/filter)
POST	/students	Add new student
GET	/students/{id}	Get one student
PUT	/students/{id}	Update student
DELETE	/students/{id}	Delete student

Fees

Method	Endpoint	Description
GET	/fees	List all invoices

POST	/fees/generate	Create challan for student
POST	/fees/{id}/pay	Mark invoice as paid
GET	/fees/{id}/pdf	Download challan PDF

Results

Method	Endpoint	Description
GET	/results/{student_id}	All marks for student
POST	/results	Enter new marks
GET	/results/{student_id}/report-card/pdf	Download report card PDF

Dashboard

Method	Endpoint	Description
GET	/dashboard/stats	Aggregate numbers + chart data

AI — At-Risk

Method	Endpoint	Description
GET	/ai/at-risk	All students with risk scores
GET	/ai/at-risk/{student_id}	One student risk detail + SHAP

Chatbot (RAG)

Method	Endpoint	Description
POST	/chatbot/ask	Ask a policy question
GET	/chatbot/topics	List topics the bot knows

7. AI Features

7.1 At-Risk Student Prediction

A **Random Forest Classifier** is trained at server startup on student data from the database. It predicts a risk score (0–100%) for every student and uses **SHAP** to explain the top 3 contributing factors in plain English.

Input Features	Model	Output
avg_marks_pct fee_defaults failing_subjects grade_trend	Random Forest Classifier (scikit-learn) + SHAP	Risk Score 0–100% ■ High >70% ■ Medium 40–70% ■ Low <40% + Top 3 reasons

Example Output: ■ Ahmed Khan — Risk: 84% | "Avg marks dropped from 72% to 51%" | "Failing 3 subjects" | "2 unpaid invoices"

7.2 RAG Policy Chatbot

RAG (Retrieval-Augmented Generation) — the chatbot searches school policy documents first, then answers using *only what it found*. It will say "I don't know" if the answer is not in the documents. This prevents hallucination.

Step	Action	Tool Used
Setup (once)	Load 5 policy .txt files Split into 200-word chunks Convert to embeddings Store in FAISS index	sentence-transformers + FAISS
Per Query	Convert question → vector FAISS finds top-3 chunks Send question + chunks to LLM Return grounded answer + source	FAISS + Groq API (free)

Knowledge Base Documents:

Knowledge Base File	Contents
fee_policy.txt	Fee due dates, late fees, refund rules, concessions
exam_policy.txt	Exam schedule, grading scale, retake rules
admission_policy.txt	Age requirements, required documents, transfer rules
conduct_rules.txt	Uniform, discipline, leave rules
academic_calendar.txt	Term dates, holidays, exam dates, events

8. 2-Day Build Schedule

■ ■ Day 1 — Backend (10 Hours)

Hour	Task	Output Files
1–2	Project setup + DB models + JWT auth	config.py, database.py, models/*.py, auth.py
3–4	Students API (full CRUD)	routers/students.py
5–6	Fee API + challan PDF generation	routers/fees.py, services/pdf_service.py
7	Results API + report card PDF	routers/results.py
8	Dashboard stats + seed 50 students	routers/dashboard.py, seed_data.py
9	At-Risk AI model + endpoint	services/ai_service.py, routers/ai.py
10	RAG Chatbot (rag_service + router + docs)	services/rag_service.py, routers/chatbot.py, knowledge_base/*.txt

■ ■ Day 2 — Frontend (10 Hours)

Hour	Task	Output Files
1–2	React setup + Layout + Login page	package.json, App.jsx, Layout.jsx, Login.jsx
3–4	Dashboard + Students pages	Dashboard.jsx, Students.jsx
5	Fees management page	Fees.jsx
6	Results entry page	Results.jsx
7	At-Risk AI page (badges + SHAP reasons)	AtRisk.jsx
8	RAG Chatbot page (chat UI)	Chatbot.jsx
9	API client — wire everything up	api/client.js
10	Polish UI + full end-to-end demo test	Final demo rehearsal

9. PDF Output Designs

Fee Challan PDF

AL-NOOR ACADEMY — Fee Challan

Student : Ahmed Khan Roll No : 045

Class : Grade 8-B Month : October 2025

Tuition Fee Rs. 3,500

Exam Fee Rs. 500

TOTAL Rs. 4,000

Due Date : 10-Oct-2025 Challan No : CHN-1045

[Bank Copy] [Student Copy]

Report Card PDF

AL-NOOR ACADEMY — Student Report Card (2025-2026)

Student : Ahmed Khan Roll : 045 Grade : 8-B

Subject	Marks Obtained	Total	Grade
English	78	100	B
Mathematics	91	100	A
Science	85	100	A
Urdu	72	100	B
Islamiat	88	100	A
Pakistan Studies	80	100	A

Total: 494 / 600 Percentage: 82.3% Grade: A

10. 5-Minute Demo Script for Judges

#	Action	Time	Impact on Judges
1	Login as admin (username: admin / password: admin123)	30 sec	Shows security
2	Dashboard — live stats + bar charts	45 sec	Shows data value
3	Add a new student via the form	30 sec	Core feature works
4	Generate fee challan → download PDF	1 min	Tangible output
5	Enter exam marks → download report card PDF	30 sec	Tangible output
6	At-Risk page → red student + 3 SHAP reasons	1 min	■ AI wow #1
7	Chatbot: "What is the fee refund policy?"	30 sec	■ AI wow #2

11. Setup Instructions

Step 1 — Get a Free Groq API Key

Go to <https://console.groq.com> → Sign up free → Create API Key → Copy it.

Step 2 — Backend Setup

```
cd G:\Edutech\backend
pip install -r requirements.txt
echo GROQ_API_KEY=paste_your_key_here > .env
python seed_data.py
uvicorn main:app --reload
# Server → http://localhost:8000
# API Docs → http://localhost:8000/docs
```

Step 3 — Frontend Setup

```
cd G:\Edutech\frontend
npm install
npm run dev
# App → http://localhost:5173
```

12. Default Demo Configuration

Setting	Default Value
School Name	Al-Noor Academy
Grades	Grade 6, 7, 8, 9, 10
Subjects	English, Urdu, Mathematics, Science, Islamiat, Pakistan Studies
Tuition Fee	Rs. 3,500 / month
Exam Fee	Rs. 500 / semester
Currency	PKR (Rs.)
Admin Username	admin
Admin Password	admin123
Demo Students	50 students seeded across all grades